

Part 1

1. Sources:

<https://www.wscubetech.com/resources/dsa/bucket-sort>

<https://pythonread.github.io/dsa/bucket-sort.html>

https://www.reddit.com/r/learnprogramming/comments/eel3g9/why_dont_we_use_bucket_sort_so_much_when_its_so/

<https://www.baeldung.com/java-bucket-sort>

<https://www.geeksforgeeks.org/dsa/bucket-sort-2/>

2. Identify 3 references that you think were the most informative/easiest to understand:

<https://pythonread.github.io/dsa/bucket-sort.html>

<https://www.wscubetech.com/resources/dsa/bucket-sort>

<https://www.geeksforgeeks.org/dsa/bucket-sort-2/>

3. Sample Code:

- *Python Code*

```

1 def bucketSort(array):
2     bucket = []
3     # Create empty buckets
4     for i in range(len(array)):
5         bucket.append([])
6     # Insert elements into their respective buckets
7     for j in array:
8         index_b = int(10 * j)
9         bucket[index_b].append(j)
10    # Sort the elements of each bucket
11    for i in range(len(array)):
12        bucket[i] = sorted(bucket[i])
13    # Get the sorted elements
14    k = 0
15    for i in range(len(array)):
16        for j in range(len(bucket[i])):
17            array[k] = bucket[i][j]
18            k += 1
19    return array
20
21 array = [.42, .32, .33, .52, .37, .47, .51]
22 print(bucketSort(array))

```

- Python Output

```
[0.32, 0.33, 0.37, 0.42, 0.47, 0.51, 0.52]
```

- Java Code

```

1 import java.util.ArrayList;
2 import java.util.Collections;
3
4 public class Main {
5
6     public static void bucketSort(float[] arr, int n) {
7         if (n <= 0) return;
8
9         @SuppressWarnings("unchecked")
10        ArrayList<Float>[] bucket = new ArrayList[n];
11
12        // Create empty buckets
13        for (int i = 0; i < n; i++)
14            bucket[i] = new ArrayList<Float>();
15
16        // Add elements into the buckets
17        for (int i = 0; i < n; i++) {
18            int bucketIndex = (int)(arr[i] * n);
19            bucket[bucketIndex].add(arr[i]);
20        }
21
22        // Sort elements of each bucket
23        for (int i = 0; i < n; i++) {
24            Collections.sort(bucket[i]);
25        }
26
27        // Concatenate results
28        int index = 0;
29        for (int i = 0; i < n; i++) {
30            for (float value : bucket[i]) {
31                arr[index++] = value;
32            }
33        }
34    }
35
36    public static void main(String[] args) {
37        float[] arr = {0.42f, 0.32f, 0.23f, 0.52f, 0.25f, 0.47f, 0.51f};
38        bucketSort(arr, arr.length);
39        for (float f : arr) System.out.print(f + " ");
40    }
41 }

```

- Python Output

```
0.23 0.25 0.32 0.42 0.47 0.51 0.52
```

4. In your opinion, why do you think this sorting algorithm is not part of our course?

I think why bucket sort were not part of our course because of the following:

- **Dependency on Data Distribution:**

Bucket sort's efficiency heavily relies on the input data being uniformly distributed over a known range. In cases of skewed or clustered data, its performance degrades significantly, potentially reaching a worst-case time complexity of $O(n^2)$, similar to less efficient algorithms like Bubble Sort. This makes it less reliable for general-purpose sorting compared to algorithms with more consistent performance across various data distributions.

- **Space Complexity:**

Bucket sort requires additional memory to store the buckets, leading to a space complexity of $O(n + k)$, where 'k' is the number of buckets. For very large datasets or when memory is a critical constraint, this additional space overhead can be a limitation.

- **Complexity of Implementation and Parameter Tuning:**

While the core idea can get a little simple if I think of it, but the effectively implementation of bucket sort can involve considerations like choosing an appropriate number of buckets and designing a suitable hash function to distribute elements evenly, which can get a little bit more complex.

- **Specialized Nature:**

Bucket sort is a specialized sorting algorithm can be useful in some specific conditions (uniform data distribution, known range, acceptable memory usage). Additionally, our Course often prioritize involving the use of more general-purpose algorithms that are widely applicable and provide a foundational understanding of different sorting paradigms, even if their average case performance might not always match bucket sort's best-case.

- **Alternative Specialized Algorithms:**

In scenarios where bucket sort might be considered, other specialized algorithms like Radix Sort or Counting Sort can often offer better performance or be more suitable for specific data types (ex. integers). These other alternatives might be introduced instead or alongside with bucket sort.